

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Today.

- 1 Check-in
- 2 What an agent actually is
- 3 Three standards
- 4 Git: one change, start to finish
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

A chat model writes. An agent **acts**.

A CHAT MODEL

Text in, text out.

It has never seen your folder.

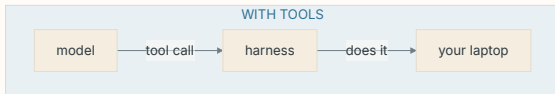
AN AGENT

The same model, with hands.

Reads. Edits. Runs. Sees the result.
Chooses again.

Agent = model + tools + a loop.

Without tools it can only give advice.



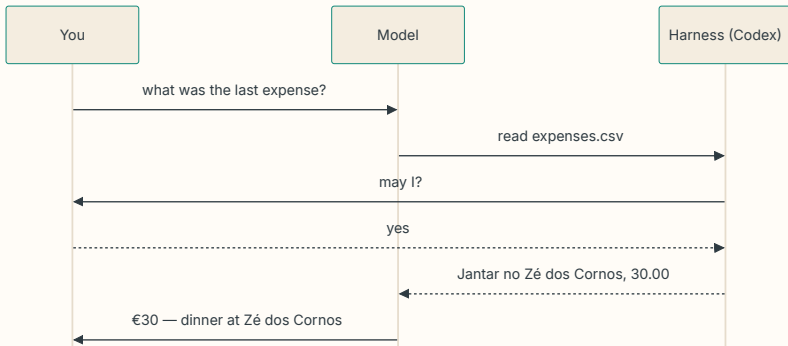
WHAT A TOOL IS

A named action the model is allowed to ask for. Read a file · edit a file · run a command.

WHY IT CHANGES EVERYTHING

The result comes back *to the model*. Without tools you are the hands — and the only one who finds out.

The model asks. The harness does.



Read · edit · run. That is the whole vocabulary.

What actually goes back and forth.

YOU TYPE what was the last expense?

THE MODEL EMITS `{"tool": "read_file", "path": "data/expenses.csv"}`

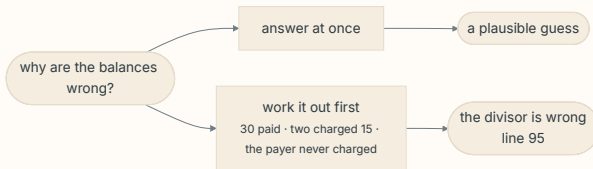
CODEx ASKS *Codex wants to read* `data/expenses.csv` *. Allow?*

CODEx PASTES BACK `29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25`

THE MODEL ANSWERS €30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked; you allowed it; the file's own line was pasted into the conversation. That line is the only reason it can answer.

Answer at once — or **work it out**.



WHAT IT IS

Work it out in steps, *then* decide — as you would on paper.

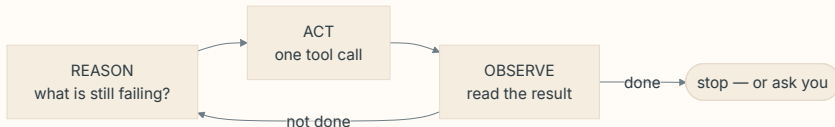
THE PROBLEM IT SOLVES

A model answers in one pass — the same effort for $2 + 2$ as for your bug. The steps it writes **are** the extra thinking.

WHY IT MATTERS HERE

A guess at step one is still wrong at step ten.

The loop has a name: **ReAct**.



THE HARNESS

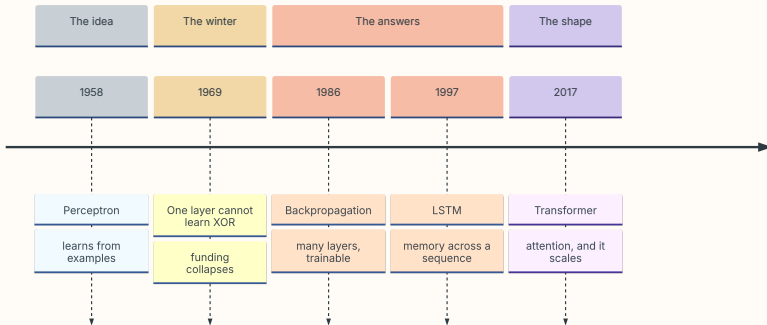
Runs the tool. Asks you. Feeds the result back. Decides when to stop. Codex is one; so are Claude Code and Cursor.

WHY YOU CARE

They all run this same loop. Learn it once, and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

The ideas came first. By decades.



Sixty years of theory. **What was missing was never the idea.**

Compute and text arrived **at the same time.**

COMPUTE

Arithmetic got cheap

2007 — graphics cards became programmable. 2012 — a network trained on two of them won the image contest outright.

DATA

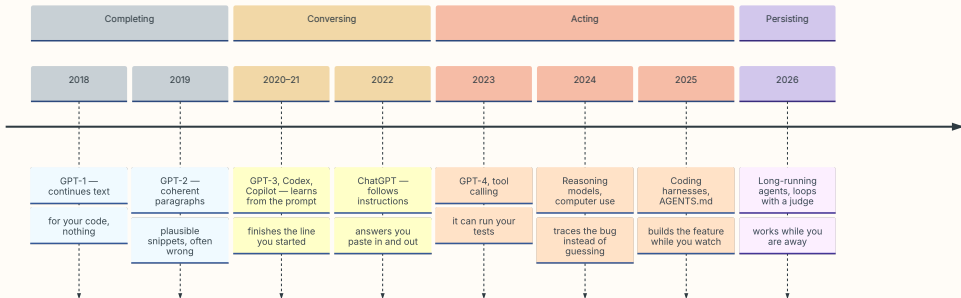
The web became a corpus

From 2007 — a non-profit crawling the public web and giving it away. Billions of pages of text, free.

Neural nets are not new. **Cheap parallel arithmetic and a scraped web are.**

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (founded 2007, public crawls from 2011)

Then eight years, four kinds of capability.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**.

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

`AGENTS.md` — read before anything else.

WHAT IT IS

A README for agents.

Every session starts by reading it.

WHAT GOES IN

What it cannot guess.

Commands. Boundaries.
How to report back.

WHAT STAYS OUT

What it can read.

Your code, explained.
Every extra line hides a real one.

It lives in git. When the agent misbehaves, fix a line here — not the prompt.

Three rules. One commit.

BOUNDARY

One branch per spec.

Never straight to `main`.

EVIDENCE

Show me the test output.

Not a sentence saying it passed.

OFF-LIMITS

Never touch the tests.

If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

Decide what **done** means. First.

IT STOPS WHEN IT LOOKS DONE

So give it a check.

Failing tests are the to-do list. Green is the finish line.

NEVER MOVE THE GOALPOSTS

Editing a test is cheating.

Not finishing.

EVIDENCE, NOT CLAIMS

Output, not adjectives.

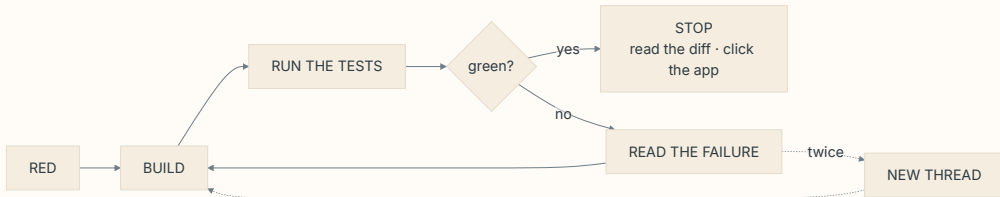
"All tests pass" is a sentence.

GREEN IS NOT DONE

Read the diff. Click the app.

App wrong, tests green? You found a missing test.

Red → **build** → **run** → **fix** → **green.**



ONE TASK PER LOOP

One bug, or one spec.

LET IT LOOP

It reads its own failures.
You read the result.

KNOW WHEN TO STOP

Green — or twice round
on the same failure. Then
a new thread.

Every good ask has four parts.

1

The task

One thing. Name the spec.

2

The boundary

This branch.
Never the tests.

3

The judge

Run the tests
until they pass.

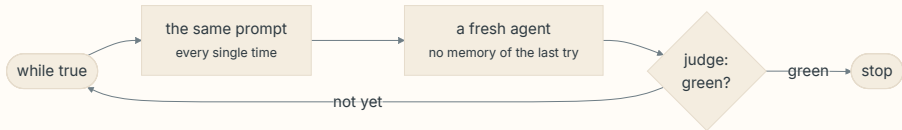
4

The evidence

The output, and
every file
touched.

Written out in full on the Week 4 page. Copy it; change two names.

Meet Ralph.



WHAT IT IS

The same prompt, handed to a fresh agent again and again, until the judge says green.

WHY IT NEEDS TODAY

Nobody is watching. The rulebook keeps it in bounds; the judge is all that can stop it.



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

This is `main`.



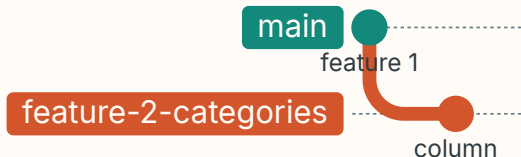
One version, one history, one truth — no folder full of *model_v3_FINAL_jm*. Everybody's copy starts from here.

You never edit the original.



A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

Every save point is a commit.



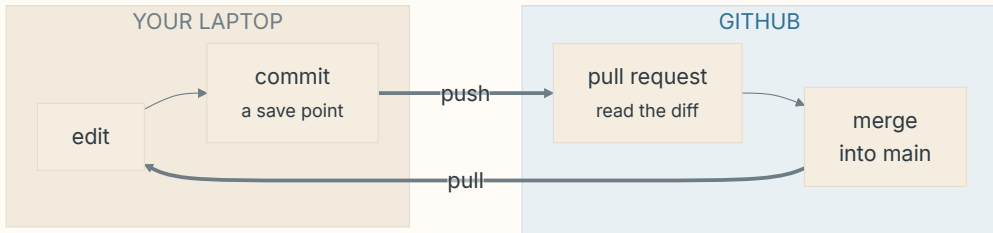
A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

`main` **has not moved.**



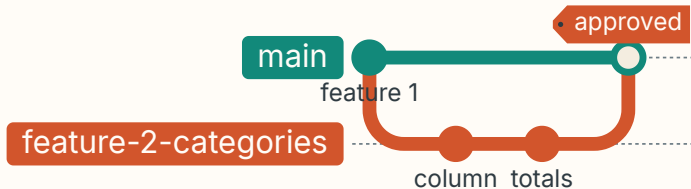
Three commits deep, half of it broken, the agent having a bad day — the version everyone else is using is exactly as you left it.

Until you push, it is only on your laptop.



Push puts your draft where the team can see it. A **pull request** is you asking someone to read the diff before it counts — today, that someone is you, reviewing the agent.

Approved. Now it is the official version.



Merge is the sign-off. `main` now includes your work — and the next person's copy starts from here.

Watch it work. Then check its work.

- 1 **Branch.** Named after the spec.
- 2 **Red first.** See what is missing before anything is written.
- 3 **New thread. Paste the ask.** Then sit on your hands.
- 4 **Watch it loop.**
- 5 **Green — now don't trust it.** Read the diff. Click the app.
- 6 **Commit, push, pull request, merge.**

One bug on `main`. One feature on a branch.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then
conversion by source.

Files, test names and the ask: Week 4 page.

Five things you will see.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, three times.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

What did it get wrong? Write the rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

That is how a rulebook grows: from what went wrong, one line at a time.

Next week you build alone.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rulebook • judge • loop.